TaskTracker — Planejamento Lógico e Arquitetural

Bootcamp II — Fase 1 (Entrega Inicial)
Aluno: João Pedro Vieira de Oliveira

1. Visão geral do problema

O TaskTracker é uma aplicação de linha de comando (CLI) para cadastro e acompanhamento de tarefas pessoais. O usuário interage por um menu de texto no terminal, podendo registrar novas tarefas, consultar as já registradas e encerrar o programa.

O objetivo da aplicação é oferecer um registro simples e confiável de compromissos, garantindo que nenhum dado inválido entre no sistema. Nesta primeira versão, os dados são mantidos apenas em memória durante a execução — a persistência em arquivo ou banco de dados fica prevista para evoluções futuras.

Escopo desta versão

Dentro do escopo	Fora do escopo
Menu de navegação contínuo no terminal	Interface gráfica ou web
Cadastro de tarefa com validação de dados	Persistência em disco ou banco de dados
Listagem de todas as tarefas registradas	Edição e exclusão de tarefas
Encerramento controlado da aplicação	Múltiplos usuários ou autenticação

2. Decomposição do problema

O problema foi dividido em unidades menores, cada uma com uma responsabilidade única. Essa separação orienta diretamente a divisão do código em funções.

#	Subproblema	Responsabilidade
1	Exibir o menu	Apresentar as três opções disponíveis ao usuário
2	Ler a opção escolhida	Capturar a entrada e direcionar para a ação correspondente
3	Validar o título	Garantir que o título não seja vazio nem composto só de espaços
4	Validar a prioridade	Aceitar exclusivamente Alta, Média ou Baixa
5	Montar e armazenar a tarefa	Reunir os campos, atribuir o status inicial e guardar na coleção
6	Listar as tarefas	Percorrer a coleção e exibir os dados de forma legível
7	Tratar a lista vazia	Informar ao usuário quando não houver nenhum registro
8	Encerrar a aplicação	Interromper o laço principal e despedir-se do usuário

3. Arquitetura de dados

3.1 Entidade Tarefa

Cada tarefa é representada por um dicionário com cinco campos:

Campo	Tipo	Obrigatório	Valor inicial	Regra de validação
`titulo`	String	Sim	—	Não pode ser vazio nem conter apenas espaços em branco
`descricao`	String	Não	—	Texto livre, sem restrição
`prioridade`	String	Sim	—	Aceita somente `Alta`, `Média` (ou `Media`) e `Baixa`
`data_limite`	String	Não	—	Data planejada, no formato `DD/MM/AAAA`
`status`	String	Sim	`Pendente`	Atribuído automaticamente no cadastro, sem entrada do usuário

3.2 Coleção de tarefas

As tarefas são armazenadas em uma **lista** (`tarefas`), escolhida por três motivos:

- preserva a ordem de inserção, o que torna a listagem previsível;
- permite acréscimo de novos itens em tempo constante;
- é percorrida de forma direta e simples na exibição.

A lista é criada vazia no início da execução e vive apenas enquanto o programa está rodando.

3.3 Constantes do sistema

Constante	Conteúdo	Finalidade
`PRIORIDADES_VALIDAS`	Mapa das formas aceitas de escrita para o rótulo oficial	Centraliza a validação da prioridade e normaliza maiúsculas, minúsculas e acentuação
`STATUS_INICIAL`	`"Pendente"`	Evita a repetição do texto no código e facilita uma alteração futura

---

4. Algoritmo em português estruturado

4.1 Programa principal

```
INÍCIO
    CRIAR lista vazia chamada tarefas
    EXIBIR mensagem de boas-vindas

    REPITA INDEFINIDAMENTE
        EXIBIR o menu com as opções 1, 2 e 3
        LER a opção digitada pelo usuário

        SE a opção for "1" ENTÃO
            EXECUTAR cadastrar_tarefa(tarefas)
        SENÃO SE a opção for "2" ENTÃO
            EXECUTAR visualizar_tarefas(tarefas)
        SENÃO SE a opção for "3" ENTÃO
            EXIBIR mensagem de encerramento
            INTERROMPER o laço
        SENÃO
            EXIBIR "Opção inválida. Escolha 1, 2 ou 3."
        FIM SE
    FIM REPITA
FIM
```

4.2 Sub-rotina: cadastrar tarefa

```
INÍCIO cadastrar_tarefa(tarefas)
```

```
    EXIBIR o cabeçalho do cadastro

    titulo     ← ler_titulo()
    LER descricao
    prioridade ← ler_prioridade()
    LER data_limite

    CRIAR tarefa com:
        titulo, descricao, prioridade, data_limite
        status ← "Pendente"

    ADICIONAR tarefa ao final da lista tarefas
    EXIBIR confirmação do cadastro
FIM
```

4.3 Sub-rotina: ler título (validação de obrigatoriedade)

```
INÍCIO ler_titulo()
    REPITA INDEFINIDAMENTE
        LER titulo
        REMOVER os espaços das extremidades do titulo

        SE titulo não estiver vazio ENTÃO
            RETORNAR titulo
        SENÃO
            EXIBIR "O título é obrigatório e não pode ficar em branco."
        FIM SE
    FIM REPITA
FIM
```

4.4 Sub-rotina: ler prioridade (validação estrita)

```
INÍCIO ler_prioridade()
    REPITA INDEFINIDAMENTE
        LER entrada
        REMOVER espaços das extremidades e CONVERTER para minúsculas

        SE entrada estiver em PRIORIDADES_VALIDAS ENTÃO
            RETORNAR o rótulo oficial correspondente
        SENÃO
            EXIBIR "Prioridade inválida. Use apenas: Alta, Media ou Baixa."
        FIM SE
    FIM REPITA
FIM
```

4.5 Sub-rotina: visualizar tarefas

```
INÍCIO visualizar_tarefas(tarefas)
    EXIBIR o cabeçalho da listagem

    SE a lista tarefas estiver vazia ENTÃO
        EXIBIR "Nenhuma tarefa cadastrada no momento."
        RETORNAR
    FIM SE

    PARA CADA tarefa na lista, com um contador começando em 1
        EXIBIR o número da tarefa
        EXIBIR título, descrição, prioridade, data limite e status
    FIM PARA

    EXIBIR o total de tarefas
FIM
```

---

5. Tratamento de erros previstos

Situação	Resposta do sistema
Opção de menu diferente de 1, 2 ou 3	Mensagem de erro e retorno ao menu, sem encerrar o programa
Título vazio ou só com espaços	Mensagem de erro e repetição do prompt até receber um valor válido

	valor válido
Prioridade fora das três opções	Mensagem de erro e repetição do prompt até receber um valor válido
Listagem com a coleção vazia	Mensagem informando que não há registros, em vez de uma tela em branco
Descrição ou data limite não informadas	Aceitas em branco; a listagem exibe um texto indicativo no lugar

---

6. Estrutura planejada do repositório

```
tasktracker-projeto/
├── README.md                    # Documentação da aplicação e instruções de uso
├── .gitignore                   # Regras para arquivos temporários do Python e ambiente virtual
├── docs/
│   └── planejamento_logico.pdf  # Este documento
└── src/
    └── main.py                  # Código-fonte principal da aplicação
```

Justificativa da organização

- `src/` isola o código-fonte dos arquivos de apoio, facilitando o empacotamento previsto para a Fase 3.
- `docs/` guarda a documentação de planejamento, preservando a rastreabilidade entre as fases do projeto.
- `README.md` e `.gitignore` ficam na raiz por convenção: são os primeiros arquivos lidos por quem chega ao repositório e pela própria ferramenta Git.

---

7. Mapeamento para Git e GitHub

7.1 Fluxo de versionamento planejado

Etapa	Comando	Resultado esperado
1. Inicialização	`git init`	Cria o repositório local e a estrutura de pastas
2. Preparação	`git add <arquivos>`	Move as alterações para a área de staging
3. Commit inicial	`git commit -m "..."`	Registra README, .gitignore e o documento de planejamento
4. Commit de desenvolvimento	`git commit -m "..."`	Registra o menu e o laço principal em `src/main.py`
5. Commit de validações	`git commit -m "..."`	Registra as regras de título e prioridade
6. Vínculo remoto	`git remote add origin <url>`	Associa o repositório local ao GitHub
7. Publicação	`git push -u origin main`	Envia o histórico completo para o repositório remoto

7.2 Padrão de mensagens de commit

As mensagens seguem o padrão *Conventional Commits*, com um prefixo que identifica a natureza da mudança:

- `chore:` — estrutura, configuração e arquivos de apoio
- `feat:` — nova funcionalidade da aplicação
- `docs:` — alterações restritas à documentação

Cada commit corresponde a uma unidade lógica de trabalho, de modo que o histórico leia como a evolução real do projeto e não como um despejo único de arquivos.

---

8. Próximas fases

Fase	Entregas previstas
Fase 2 — Intermediária	Estruturação do repositório, versionamento com Git/GitHub, implementação em Python e validação das regras de negócio
Fase 3 — Final	Empacotamento da aplicação, containerização com Docker, testes e deploy